



Python Programming

Introduction



- Guido van Rossum developed Python, a high-level, general-purpose programming language.
- Multi-paradigm: object-oriented, procedural, functional, structured, reflective
- First release was in Feb, 1991 and it is primarily written in C.
- Rated as the most popular language in 2024.

Key features

- 1.
2. Interpreted language
3. Readability and simplicity
4. Dynamic typing yet strong typing
5. Built-in object types
6. Built-in tools
7. Library utilities

Key features

8. Extensive standard and 3rd party libraries
9. Cross-platform compatibility
10. Large and active community
11. Automatic memory management
12. Programming-in-the-large support

Building blocks

Identifiers

Keywords

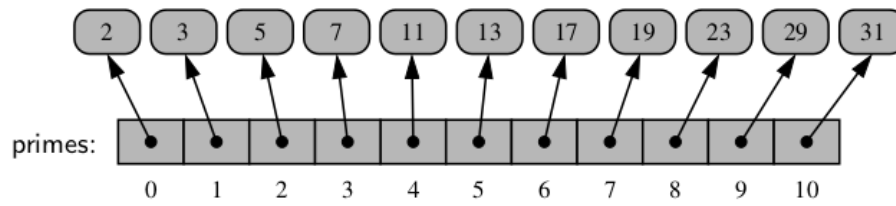
Reserved Words									
False	as	continue	else	from	in	not	return	yield	
None	assert	def	except	global	is	or	try		
True	break	del	finally	if	lambda	pass	while		
and	class	elif	for	import	nonlocal	raise	with		

Building blocks

Commonly used built-in classes

Class	Description	Immutable?
bool	Boolean value	✓
int	integer (arbitrary magnitude)	✓
float	floating-point number	✓
list	mutable sequence of objects	
tuple	immutable sequence of objects	✓
str	character string	✓
set	unordered set of distinct objects	
frozenset	immutable form of set class	✓
dict	associative mapping (aka dictionary)	

Lists



Identifiers

- Identifiers in Python are case-sensitive, so temperature and Temperature are distinct names.
- Identifiers can be composed of almost any combination of letters, numerals, and underscore characters.
- An identifier cannot begin with a numeral and that there are 33 specially reserved words that cannot be used as identifiers:

Reserved Words								
False	as	continue	else	from	in	not	return	yield
None	assert	def	except	global	is	or	try	
True	break	del	finally	if	lambda	pass	while	
and	class	elif	for	import	nonlocal	raise	with	

Types

- Python is a **dynamically typed** language, as there is no advance declaration associating an identifier with a particular data type.
- An identifier can be associated with any type of object, and it can later be reassigned to another object of the same (or different) type.
- Although an identifier has no declared type, the object to which it refers has a definite type.

9

Objects

The process of creating a new instance of a class is known as **instantiation**.

To instantiate an object we usually invoke the constructor of a class

```
w = Widget()
```

This is assuming that the constructor does not require any parameters.

If the constructor does require parameters, we might use a syntax such as

```
w = Widget(a, b, c)
```

Many of Python's built-in classes use a **literal form** for designating new instances. For ex.

```
percent = 65.7.
```

10

Calling Methods

- Python supports functions a syntax such as `max(data)`, in which case `data` is a parameter sent to the function.
- Python's classes may also define one or more methods (also known as member functions), which are invoked on a specific instance of a class using the dot ("`.`") operator.
- For example, Python's list class has a method named `sort` that can be invoked with a syntax such as `data.sort( )`. (this method rearranges the contents of the list)

Built-In Classes

Class	Description	Immutable?
bool	Boolean value	✓
int	integer (arbitrary magnitude)	✓
float	floating-point number	✓
list	mutable sequence of objects	
tuple	immutable sequence of objects	✓
str	character string	✓
set	unordered set of distinct objects	
frozenset	immutable form of <code>set</code> class	✓
dict	associative mapping (aka dictionary)	

A class is *immutable* if each object of that class has a fixed value upon instantiation that cannot subsequently be changed. For example, the **float** class is immutable.

The bool Class

The **bool** class is used for logical (Boolean) values, and the only two instances of that class are expressed as the literals:

True & False

The default constructor, `bool()`, returns `False`.

Python allows the creation of a Boolean value from a non-boolean type using the syntax `bool(foo)` for value `foo`. The interpretation depends upon the type of the parameter.

- Numbers evaluate to `False` if zero, and `True` if nonzero. [13](#)

The int Class

The **int** class is designed to represent integer values with arbitrary magnitude.

Python automatically chooses the internal representation for an integer based upon the magnitude of its value.

The integer constructor, `int()`, returns 0 by default.

This constructor can also construct an integer value based upon an existing value of another type.

For example, if `f` represents a floating-point value, the syntax `int(f)` produces the truncated value of `f`. For example, `int(3.35)` produces the value 3, while `int(-3.1)` produces the value -3.

The constructor can also be used to parse a string that represents an integer. For example, the expression `int('345')` produces the integer value 345.

The float Class

The **float** class is the floating-point type in Python.

The floating-point equivalent of an integral number, 2, can be expressed directly as 2.0.

One other form of literal for floating-point values uses scientific notation. For example, the literal 6.022e23 represents the mathematical value 6.022×10^{23} .

The constructor `float( )` returns 0.0.

When given a parameter, the constructor, `float`, returns the equivalent floating-point value.

`float(2)` returns the floating-point value 2.0

`float('3.14')` returns 3.14

15

The list Class

A **list** instance stores a sequence of objects, that is, a sequence of references (or pointers) to objects in the list.

Elements of a list may be arbitrary objects (including the `None` object).

Lists are array-based sequences and a list of length `n` has elements indexed from 0 to `n-1` inclusive.

Lists have the ability to dynamically expand and contract their capacities as needed.

Python uses the characters `[ ]` as delimiters for a list literal.

`[]` is an empty list.

`['red', 'green', 'blue']` is a list containing three string instances.

The `list( )` constructor produces an empty list by default.

The list constructor will accept any iterable parameter.

`list('hello')` produces a list of individual characters, `['h', 'e', 'l', 'l', 'o']`.

16

The tuple Class

The **tuple** class provides an immutable (unchangeable) version of a sequence, which allows instances to have an internal representation that may be more streamlined than that of a list. Parentheses delimit a tuple.

The empty tuple is (). eg `a = ()`/`a = tuple()`

To express a tuple of length one as a literal, a comma must be placed after the element, but within the parentheses.

Eg. `(23,)` is a one-element tuple.

17

The str Class

String literals can be enclosed in single quotes, as in 'hello', or double quotes, as in "hello".

A string can also begin and end with three single or double quotes, if it contains newlines in it.

```
print("""Welcome to the GPA calculator.  
Please enter all your letter grades, one per line.  
Enter a blank line to designate the end.""")
```

18

The set Class

A collection of elements, without duplicates, and without an inherent order to those elements.

Only instances of immutable types can be added to a Python set. Therefore, objects such as integers, floating-point numbers, and character strings are eligible to be elements of a set.

19

The set Class

Python uses curly braces { and } as delimiters for a set

Eg. as {23} or {'red', 'green', 'blue'}

The exception to this rule is that { } does not represent an empty set. Instead, the constructor set() returns an empty set.

The frozenset class is an immutable form of the set type, itself.

20

The dict Class

Python's dict class represents a dictionary, or mapping, from a set of distinct keys to associated values.

Python implements a dict using an almost identical approach to that of a set, but with storage of the associated values.

The literal form `{ }` produces an empty dictionary.

A nonempty dictionary is expressed using a comma-separated series of key:value pairs. For example, the dictionary `{'ga' : 'Irish', 'de' : 'German'}` maps 'ga' to 'Irish' and 'de' to 'German'.

Alternatively, the constructor accepts a sequence of key-value pairs as a parameter, as in `dict(pairs)` with `pairs = [('ga', 'Irish'), ('de', 'German')]`.

21

Expressions and Operators

- Existing values can be combined into expressions using special symbols and keywords known as operators.
- The semantics of an operator depends upon the type of its operands.
- Eg. when `a` and `b` are numbers, the syntax `a + b` indicates addition, while if `a` and `b` are strings, the operator `+` indicates concatenation.

22

Logical Operators

Python supports the following keyword operators for Boolean values:

not	unary negation
and	conditional and
or	conditional or

The **and** and **or** operators **short-circuit**, in that they do not evaluate the second operand if the result can be determined based on the value of the first operand.

23

Equality Operators

Python supports the following operators to test two notions of equality:

is	same identity
is not	different identity
==	equivalent
!=	not equivalent

The expression, `a is b`, evaluates to `True`, precisely when identifiers `a` and `b` are aliases for the same object.

The expression `a == b` tests a more general notion of equivalence.

24

Comparison Operators

Data types may define a natural order via the following operators:

<code><</code>	less than
<code><=</code>	less than or equal to
<code>></code>	greater than
<code>>=</code>	greater than or equal to

These operators have expected behavior for numeric types, and are defined lexicographically, and case-sensitively, for strings.

25

Arithmetic Operators

Python supports the following arithmetic operators:

<code>+</code>	addition
<code>-</code>	subtraction
<code>*</code>	multiplication
<code>/</code>	true division
<code>//</code>	integer division
<code>%</code>	the modulo operator

- For addition, subtraction, and multiplication, if both operands have type `int`, then the result is an `int`; if one or both operands have type `float`, the result is a `float`.
- True division is always of type `float`, integer division is always `int` (with the result truncated)

26

Bitwise Operators

Python provides the following bitwise operators for integers:

<code>~</code>	bitwise complement (prefix unary operator)
<code>&</code>	bitwise and
<code> </code>	bitwise or
<code>^</code>	bitwise exclusive-or
<code><<</code>	shift bits left, filling in with zeros
<code>>></code>	shift bits right, filling in with sign bit

27

Sequence Operators

Each of Python's built-in sequence types (str, tuple, and list) support the following operator syntaxes:

<code>s[j]</code>	element at index j
<code>s[start:stop]</code>	slice including indices [start,stop)
<code>s[start:stop:step]</code>	slice including indices start, start + step, start + 2*step, ..., up to but not equalling or stop
<code>s + t</code>	concatenation of sequences
<code>k * s</code>	shorthand for <code>s + s + s + ...</code> (k times)
<code>val in s</code>	containment check
<code>val not in s</code>	non-containment check

28

Sequence Comparisons

Sequences define comparison operations based on lexicographic order, performing an element by element comparison until the first difference is found.

For example, `[5, 6, 9] < [5, 7]` because of the entries at index 1.

<code>s == t</code>	equivalent (element by element)
<code>s != t</code>	not equivalent
<code>s < t</code>	lexicographically less than
<code>s <= t</code>	lexicographically less than or equal to
<code>s > t</code>	lexicographically greater than
<code>s >= t</code>	lexicographically greater than or equal to

29

Operators for Sets

Sets and frozensets support the following operators:

<code>key in s</code>	containment check
<code>key not in s</code>	non-containment check
<code>s1 == s2</code>	<code>s1</code> is equivalent to <code>s2</code>
<code>s1 != s2</code>	<code>s1</code> is not equivalent to <code>s2</code>
<code>s1 <= s2</code>	<code>s1</code> is subset of <code>s2</code>
<code>s1 < s2</code>	<code>s1</code> is proper subset of <code>s2</code>
<code>s1 >= s2</code>	<code>s1</code> is superset of <code>s2</code>
<code>s1 > s2</code>	<code>s1</code> is proper superset of <code>s2</code>
<code>s1 s2</code>	the union of <code>s1</code> and <code>s2</code>
<code>s1 & s2</code>	the intersection of <code>s1</code> and <code>s2</code>
<code>s1 - s2</code>	the set of elements in <code>s1</code> but not <code>s2</code>
<code>s1 ^ s2</code>	the set of elements in precisely one of <code>s1</code> or <code>s2</code>

30

Operators for Dictionaries

The supported operators for objects of type dict are as follows:

<code>d[key]</code>	value associated with given key
<code>d[key] = value</code>	set (or reset) the value associated with given key
<code>del d[key]</code>	remove key and its associated value from dictionary
<code>key in d</code>	containment check
<code>key not in d</code>	non-containment check
<code>d1 == d2</code>	d1 is equivalent to d2
<code>d1 != d2</code>	d1 is not equivalent to d2

31

Operator Precedence

Operator Precedence		
	Type	Symbols
1	member access	<code>expr.member</code>
2	function/method calls container subscripts/slices	<code>expr(...)</code> <code>expr[...]</code>
3	exponentiation	<code>**</code>
4	unary operators	<code>+expr</code> , <code>-expr</code> , <code>~expr</code>
5	multiplication, division	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>
6	addition, subtraction	<code>+</code> , <code>-</code>
7	bitwise shifting	<code><<</code> , <code>>></code>
8	bitwise-and	<code>&</code>
9	bitwise-xor	<code>^</code>
10	bitwise-or	<code> </code>
11	comparisons containment	<code>is</code> , <code>is not</code> , <code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> <code>in</code> , <code>not in</code>
12	logical-not	<code>not expr</code>
13	logical-and	<code>and</code>
14	logical-or	<code>or</code>
15	conditional	<code>val1 if cond else val2</code>
16	assignments	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , etc.

32

Program Structure

- Common to all control structures, the colon character is used to delimit the beginning of a block of code that acts as a body for a control structure.
- However, a body is more typically typeset as an indented block starting on the line following the colon.
- Python relies on the indentation level to designate the extent of that block of code, or any nested blocks of code within.

33

Conditionals

```
if first_condition:  
    first_body  
elif second_condition:  
    second_body  
elif third_condition:  
    third_body  
else:  
    fourth_body
```

34

Loops

While loop:

```
while condition:  
    body
```

For loop:

```
for element in iterable:  
    body
```

body may refer to 'element' as an identifier

Indexed For loop:

```
big_index = 0  
for j in range(len(data)):  
    if data[j] > data[big_index]:  
        big_index = j
```

35

Break and Continue

Python supports a **break** statement that immediately terminate a while or for loop when executed within its body.

```
found = False  
for item in data:  
    if item == target:  
        found = True  
        break
```

Python also supports a **continue** statement that causes the current iteration of a loop body to stop, but with subsequent passes of the loop proceeding as expected.

36

Functions

Functions are defined using the keyword **def**.

```
def count(data, target):  
    n = 0  
    for item in data:  
        if item == target:  
            n += 1  
    return n
```

count is now a new identifier which is the name of the function and it establishes the number of parameters that it expects, which defines the function's *signature*.

37

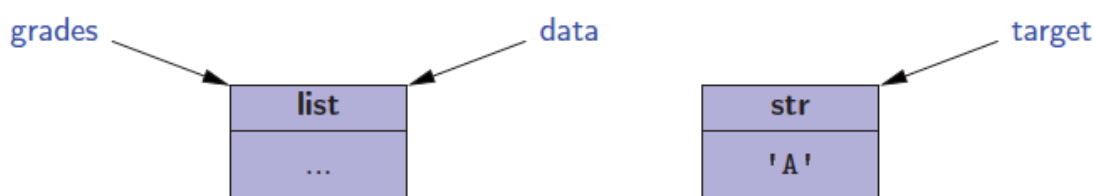
Information Passing

Parameter passing in Python follows the semantics of the standard assignment statement.

For example `prizes = count(grades, 'A')`

is the same as

```
data = grades  
target = 'A'
```



38

Simple Output

The built-in function, **print**, is used to generate standard output to the console.

In its simplest form, it prints an arbitrary sequence of arguments, separated by spaces, and followed by a trailing newline character.

Eg. the command `print('hello', 5)` outputs the string `'hello 5\n'`.

A nonstring argument `x` will be displayed as `str(x)`.

39

Simple Input

The primary means for acquiring information from the user console is a built-in function named **input**.

This function displays a prompt, if given as an optional parameter, and then waits until the user enters some sequence of characters followed by the return key.

The return value of the function is the *string of characters* that were entered strictly before the return key.

Such a string can immediately be converted, of course:

```
year = int(input('In what year were you born? '))
```

40

A Simple Program

Here is a simple program that does some input and output:

```
name = str(input("enter your name "))
percent = float(input("enter your percentage "))

print('name is ',name)
print('percentage is ',percent)

print(f'% of {name} is {percent}')
```

 - (eg. of F-strings in Python)

41

Files

Files are opened with a built-in function, **open**, that returns an object for the underlying file.

For example, the command, `fp = open('sample.txt')`, attempts to open a file named `sample.txt`.

Methods for files:

Calling Syntax	Description
<code>fp.read()</code>	Return the (remaining) contents of a readable file as a string.
<code>fp.read(k)</code>	Return the next k bytes of a readable file as a string.
<code>fp.readline()</code>	Return (remainder of) the current line of a readable file as a string.
<code>fp.readlines()</code>	Return all (remaining) lines of a readable file as a list of strings.
<code>for line in fp:</code>	Iterate all (remaining) lines of a readable file.
<code>fp.seek(k)</code>	Change the current position to be at the k^{th} byte of the file.
<code>fp.tell()</code>	Return the current position, measured as byte-offset from the start.
<code>fp.write(string)</code>	Write given string at current position of the writable file.
<code>fp.writelines(seq)</code>	Write each of the strings of the given sequence at the current position of the writable file. This command does <i>not</i> insert any newlines, beyond those that are embedded in the strings.
<code>print(..., file=fp)</code>	Redirect output of print function to the file.

42

File Read examples

```
fp = open("log.txt", "r")
content = fp.read()
print(content)
fp.close()
```

No file close is not required in this version, it is implicitly done by Python

```
with open("log.txt", "r") as fp:
    content = fp.read()
    print(content)
```

Conditional Expressions

Python supports a conditional expression syntax that can replace a simple control structure.

The general syntax is an expression of the form:

expr1 if condition else expr2

This compound expression evaluates to *expr1* if the condition is true, and otherwise evaluates to *expr2*.

Eg.

```
param = n if n >= 0 else -n    # pick the appropriate value
result = foo(param)           # call the function
```

Or even

```
result = foo(n if n >= 0 else -n)
```

Comprehension Syntax

A very common programming task is to produce one series of values based upon the processing of another series.

Often, this task can be accomplished quite simply in Python using what is known as a comprehension syntax.

[*expression* for *value* in *iterable* if *condition*]

This is the same as

```
result = [ ]
for value in iterable:
    if condition:
        result.append(expression)
```

45

Comprehension Syntax

To display a list of factors for an integer n

```
factors = [k for k in range(1,n+1) if n % k == 0]
```

To display a list of squares of first n integers using various comprehensions:

[k*k for k in range(1, n+1)]	list comprehension
{ k*k for k in range(1, n+1) }	set comprehension
(k*k for k in range(1, n+1))	generator comprehension
{ k : k*k for k in range(1, n+1) }	dictionary comprehension

46

Packing

If a series of comma-separated expressions are given in a larger context, they will be treated as a single tuple, even if no enclosing parentheses are provided.

Eg.

```
data = 2, 4, 6, 8
```

This results in identifier, data, being assigned to the tuple (2, 4, 6, 8). This behavior is called **automatic packing** of a tuple.

47

Unpacking

As a dual to the packing behavior, Python can automatically unpack a sequence, allowing one to assign a series of individual identifiers to the elements of sequence.

Eg. we can write

```
a, b, c, d = range(7, 11)
```

This has the effect of assigning a=7, b=8, c=9, and d=10.

48

Modules and Packages

- Python *modules* “package program code and data for reuse.” (Lutz)

Similar to *library* in C, *package* in Java.

- Python *packages* are hierarchical modules (i.e., modules that contain other modules).

Must contain an: `__init__.py` file to be recognized as a package.

Eg. numpy, pandas, matplotlib.

- Three commands for accessing modules/packages:

`import`

`from...import`

`reload`

Modules and Packages

mypackage/

`__init__.py`

`m1.py`

`m2.py`

subpackage/

`__init__.py`

`m3.py`

- Three commands for accessing modules:

`import`

`from...import`

`reload`

Modules and Packages:

import

The *import* command loads a module:

```
# Load the regular expression module
>>> import re
```

To access the contents of a module, use *dotted names*:

```
# Use the search method from the re module
>>> re.search('\w+', str)
```

To list the contents of a module, use *dir*:

```
>>> dir(re)
['DOTALL', 'I', 'IGNORECASE', ...]
```

Modules and Packages:

from ...import

The *from...import* command loads individual functions and objects from a module:

```
# Load the search function from the re module
>>> from re import search
```

Once an individual function or object is loaded with *from...import*, it can be used directly:

```
# Use the search method from the re module
>>> search ('\w+', str)
```

Import vs. from...import

Import

Keeps module functions separate from user functions.

Requires the use of dotted names.

Works with `reload`.

from...import

Puts module functions and user functions together.

More convenient names.

Does not work with `reload`.

Modules and Packages: **reload**

If you edit a module, you must use the `reload` command before the changes become visible in Python:

```
>>> import mymodule
...
>>> reload (mymodule)
```

The `reload` command only affects modules that have been loaded with `import`; it does not update individual functions and objects loaded with `from...import`.

Existing Modules

Some useful existing modules include the following:

Existing Modules	
Module Name	Description
array	Provides compact array storage for primitive types.
collections	Defines additional data structures and abstract base classes involving collections of objects.
copy	Defines general functions for making copies of objects.
heapq	Provides heap-based priority queue functions (see Section 9.3.7).
math	Defines common mathematical constants and functions.
os	Provides support for interactions with the operating system.
random	Provides random number generation.
re	Provides support for processing regular expressions.
sys	Provides additional level of interaction with the Python interpreter.
time	Provides support for measuring time, or delaying a program.

55

Exception Handling

- Exceptions are unexpected events that occur during the execution of a program.
- An exception might result from a logical error or an unanticipated situation.
- In Python, exceptions (also known as errors) are objects that are raised (or thrown) by code that encounters an unexpected circumstance.

The Python interpreter can also raise an exception.

- A raised error may be caught by a surrounding context that “handles” the exception in an appropriate fashion. If uncaught, an exception causes the interpreter to stop executing the program and to report an appropriate message to the console.

56

Common Exceptions

Python includes a rich hierarchy of exception classes that designate various categories of errors

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax <code>obj.foo</code> , if <code>obj</code> has no member named <code>foo</code>
EOFError	Raised if “end of file” reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by <code>next(iterator)</code> if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., <code>sqrt(-5)</code>)
ZeroDivisionError	Raised when any division operator used with 0 as divisor

57

Raising an Exception

An exception is thrown by executing the `raise` statement, with an appropriate instance of an exception class as an argument that designates the problem.

Eg. if a function for computing a square root is sent a negative value as a parameter, it can raise an exception with the command:

```
raise ValueError('x cannot be negative')
```

58

Catching an Exception

In Python, exceptions can be tested and caught using a try-except control structure.

```
try:
    ratio = x / y
except ZeroDivisionError:
    ... do something else ...
```

In this structure, the “try” block is the primary code to be executed.

Although it is a single command in this example, it can more generally be a larger block of indented code.

Following the try-block are one or more “except” cases, each with an identified error type and an indented block of code that should⁵⁹

Iterators

- Basic container types, such as list, tuple, and set, qualify as iterable types, which allows them to be used as an iterable object in a for loop.

for *element* **in** *iterable*:

- An iterator is an object that manages an iteration through a series of values. If variable, **i**, identifies an iterator object, then each call to the built-in function, **next(i)**, produces a subsequent element from the underlying series, with a **StopIteration** exception raised to indicate that there are no further elements.
- An iterable is an object, **obj**, that produces an iterator via the syntax **iter(obj)**.

Generators

The most convenient technique for creating iterators in Python is through the use of generators.

A generator is implemented with a syntax that is very similar to a function, but instead of returning values, a yield statement is executed to indicate each element of the series.

For example, a generator for the factors of n:

```
def factors(n):           # generator that computes factors
    for k in range(1,n+1):
        if n % k == 0:    # divides evenly, thus k is a factor
            yield k       # yield this factor as next result
```

61

Java Vs Python

(verbose vs conciseness)

JAVA

```
int    myCounter = 0;
String myString = String.valueOf(myCounter);
if (myString.equals("0")) ...
```

```
// print the integers from 1 to 9
for (int i = 1; i < 10; i++)
{
    System.out.println(i);
}
```

PYTHON

```
myCounter = 0
myString = str(myCounter)
if myString == "0": ...
```

```
print the integers from 1 to 9
for i in range(1,10):
    print i
```

Java Vs Python

(verbose vs conciseness)

```
int condition=10;

//if
if(condition>10)
    System.out.println("> 10");
else
    System.out.println("<= 10");

//while
while(condition>1){
    System.out.println(condition);
    condition--;
}

//switch
switch(condition){
case 1:
    System.out.println("is 1");
    break;
case 2:
    System.out.println("is 2");
    break;
}

//for
for(int i=0; i<10; i++){
    System.out.println(i);
}
```

```
condition=10;

# if
if condition > 10:
    print ">10";
elif condition == 10:
    print "=10";
else:
    print "<10";

#while
while condition > 1:
    print condition;
    condition = condition-1;

#switch
def f(x):
    return {
        1 : 1,
        2 : 2,
    }[x]
print f(condition);

#for
for x in range(1,10):
    print x;
```

Applications

- System Programming
- GUIs
- Internet Scripting
- Database Programming
- Component Integration
- Rapid Prototyping
- Numeric and Scientific Programming
- Games, Images, AI, XML, and more

... List is literally endless

References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser